

Aula 02: Prototipação

Construção de algoritmos e estruturas de controle

Maurício da Silva Pires
mspires@id.uff.br

TCC00326 - Programação de computadores
Niterói, Rio de Janeiro

Recapitulando

- ⌋ **Algoritmo** é uma sequência finita de passos simples e precisos para executar uma tarefa ou resolver um problema
 - * Nem todo passo ou tarefa é considerado simples
 - * Não há espaço para ambiguidades
 - * O computador executa os passos na exata ordem em que foram dados
- ⌋ Todo algoritmo é composto por 3 partes essenciais
 - * **Entrada:** Os dados iniciais que o algoritmo recebe
 - * **Processamento:** As instruções e cálculos executados
 - * **Saída:** O resultado final gerado após o processamento

- ⌋ A coerência lógica dos passos é extremamente importante para a obtenção dos resultados corretos
 - * Embora, em geral, existam mais de uma estratégia para resolver um problema, nem toda sequência de passos produz a saída esperada
 - * Conheça os comandos básicos e as estruturas de controle
 - * Programar é fazer um bom uso destes elementos
- ⌋ Muitas vezes chamado de “lógica de programação”, o encadeamento lógico dos comandos se refere à compreensão de que determinadas tarefas precisam ser executadas antes de outras
 - * Não se atravessa uma porta fechada (a menos que você seja a Lince Negra dos X-men)

Unidade Central de Processamento

☾ Afinal, o que são os comandos básicos de um computador?

- * Para responder a isso, precisamos entender brevemente como funciona um computador
- * Todo o processamento do computador é feito pela **Unidade Central de Processamento** (CPU)
- * O componente da CPU que efetivamente “trabalha”, executa instruções é a **Unidade Lógica e Aritmética** (ULA)

☾ Em essência, o computador é capaz de

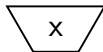
- * Armazenar e mover dados na memória
- * Executar operações lógicas e aritméticas
- * Realizar comparações matemáticas

Comandos básicos

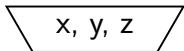
- ⌋ Os comandos básicos do computador se dividem em duas categorias
 - * Instruções de entrada e saída de dados (I/O)
 - * Atribuição de dados
- ⌋ Nas instruções I/O, o programa ou recebe dados do usuário (via teclado, mouse, tela *touch*, *scanner*) ou envia dados para o usuário (via tela, impressora, caixas de som, fone de ouvido)
- ⌋ As atribuições são o ato de armazenar informações em regiões de memória que chamamos de **variáveis**
 - * Dados de entrada precisam ser armazenados em variáveis
 - * Resultados de operações e expressões lógicas e aritméticas podem ser armazenados em variáveis

⌋ A captação típica de dados é oriunda do teclado

- * Todo dado que vem do teclado precisa ser armazenado em algum lugar
- * Em pseudocódigo, a instrução **ler(x)** significa ler algo do teclado e armazenar na variável x
- * Em fluxograma, esta mesma instrução é representada pelo seguinte desenho



⌋ Usualmente, cada dado é lido em uma única linha. Se três dados em uma linha devem ser lidos, usamos **ler(x,y,z)** em pseudocódigo e em fluxograma usamos

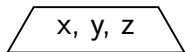


⌋ A exibição típica de dados é no monitor/tela

- * Exibe-se o dado que estava numa variável ou diretamente o dado de texto entre aspas
- * Em pseudocódigo, a instrução **exibir(x)** ou **exibir("texto")** significa imprimir na tela o conteúdo da variável *x* ou o dado "texto"
- * Em fluxograma, esta mesma instrução é representada pelo seguinte desenho



- ### ⌋ Usualmente, cada dado é impresso em uma única linha. Se três dados devem ser impressos em uma linha, usamos **exibir(x,y,z)** em pseudocódigo e em fluxograma usamos



Atribuição

- ﴿ Perceba que, no caso do comando de leitura, a atribuição do dado captado no teclado a uma região de memória é parte da instrução
- ﴿ A atribuição permite que armazenemos o resultado de alguma computação intermediária durante o processamento dos dados e resolução do problema
- ﴿ Em pseudocódigo, usamos o símbolo $\leftarrow$ para dizer que o valor que está à direita do símbolo deve ser armazenado na região de memória simbolizada pelo nome à esquerda do símbolo
- ﴿ Em fluxograma, seria

soma $\leftarrow$ 0

Controle de fluxo

- ☾ Com instruções básicas e algumas operações lógicas e aritméticas **NÃO** é possível construir bons programas de computador
 - * Precisamos ser capazes de utilizar estruturas condicionais e estruturas de repetição
 - * Estas estruturas são responsáveis por modificar o **fluxo de execução** do programa
- ☾ **Fluxo de execução** é a ordem exata em que as instruções de um programa são lidas e computadas
 - * Um programa, tipicamente, não possui um fluxo único de execução
 - * A existência de diferentes fluxos se explica pelo fato de um programa poder ser interpretado como uma função que mapeia uma entrada em uma saída

- ⌋ Embora os algoritmos sejam uma sequência finita de instruções que devem ser executadas, há algumas situações em que manter um fluxo linear de execução não é viável
 - * E se precisarmos tomar atitudes diferentes caso um valor seja par ou ímpar?
 - * E se eu precisar executar uma mesma tarefa por um certo número finito de vezes, mas eu não sei exatamente quantas vezes?
- ⌋ É importante ressaltar que toda estrutura de controle de fluxo está, em diferentes medidas, ligada ao resultado de uma comparação, ou seja, um **valor-verdade**

- ⌋ Trabalhamos com noções de lógica bivaloradas, ou seja, que possui apenas dois valores-verdade
 - * **True**, o verdadeiro, e **False**, o falso
 - * O resultado de uma comparação é um valor-verdade
 - * O resultado de uma operação lógica é um valor-verdade
- ⌋ As comparações típicas que temos em linguagens de programação são igualdade, desigualdade, maior que, maior ou igual a, menor que e menor ou igual a, que são tipicamente aplicadas apenas a números
- ⌋ As operações lógicas típicas são a negação **not**, a conjunção **and** e a disjunção **or**

Tabela-verdade

- ☾ Do ponto de vista da programação, a tabela-verdade é o guia do resultado das operações lógicas sobre valores lógicos

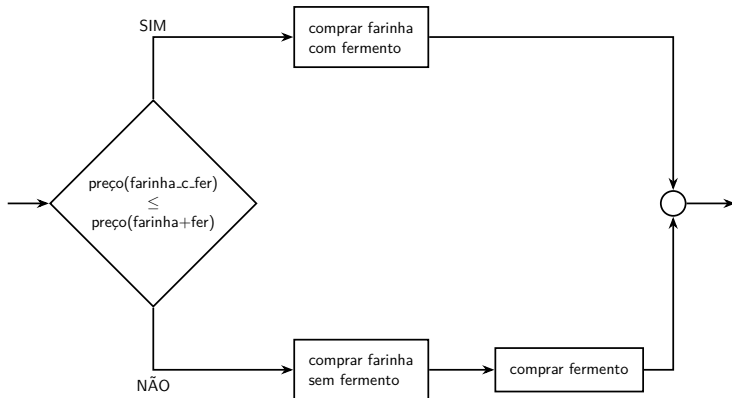
A	B	not A	A and B	A or B
True	True	False	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	False

- * A negação inverte o valor-verdade
- * A conjunção é verdadeira se ambos forem verdade
- * A disjunção é verdadeira se ao menos um é verdade

- Imagine que você deseja fazer um bolo de chocolate
- A receita que você está utilizando te diz que você pode usar farinha de trigo com fermento ou farinha de trigo sem fermento e fermento químico seco. Qual utilizar?

- ⌋ Imagine que você deseja fazer um bolo de chocolate
- ⌋ A receita que você está utilizando te diz que você pode usar farinha de trigo com fermento ou farinha de trigo sem fermento e fermento químico seco. Qual utilizar?
 - * Se não temos nenhuma das opções em casa, vamos usar aquela opção que for mais barata (*This is capitalism!*)
 - * Note que é uma comparação entre os preços que controla o fluxo de execução das nossas ações
 - * Nossa comparação será “o preço da farinha de trigo com fermento é menor que a soma dos preços da farinha de trigo sem fermento e do fermento”

Desvio condicional em fluxograma



- Usaremos um losango para indicar que a análise de um valor-verdade está acontecendo

Desvio condicional em pseudocódigo

- ⌞ O código a seguir foi adaptado por questão de espaço disponível para escrita no slide

```
...  
x <- preço da farinha com fermento  
y <- preço da farinha + preço do fermento  
se x <= y então  
    comprar farinha com fermento  
senão  
    comprar farinha e fermento  
fim-se-senão  
...
```

- ⌞ A palavra reservada **fim-se-senão** serve para marcar o encerramento do bloco de comandos da parte condicional

Tipos de desvio condicional

- ⌋ Embora, de um ponto de vista prático, exista apenas uma única instrução de desvio condicional, as linguagens de programação modernas costumam empregar 3 possíveis variações de desvio condicional
 - * SE-ENTÃO-SENÃO
 - * SE-ENTÃO (cenário em que o senão seria não fazer nada)
 - * ESCOLHA-CASO
- ⌋ O desvio do tipo SE-ENTÃO-SENÃO é o tipo de desvio condicional básico e com ele podemos simular os outros dois

Condicional SE-ENTÃO-SENÃO

```
se <condição> então
    <bloco de comandos 1>
senão
    <bloco de comandos 2>
fim-se-senão
```

- ⌞ Se a **condição** expressa for satisfeita, a execução deve seguir para as instruções do **bloco de comandos 1**. Ao terminar a execução do bloco, o fluxo de execução segue para instrução após o delimitador de escopo fim-se-senão
- ⌞ Caso a **condição** não seja satisfeita, o **bloco de comandos 2** será executado. Ao terminar a execução do bloco, o fluxo vai para a instrução após o delimitador

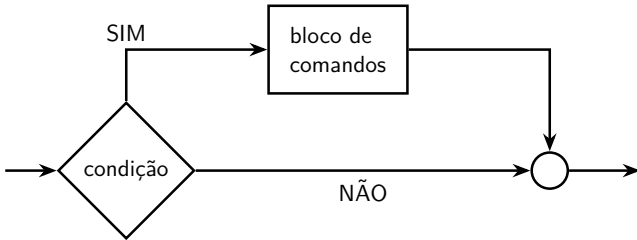
Condicional SE-ENTÃO

```
se <condição> então  
    <bloco de comandos>  
fim-se
```

- ⌞ Se a **condição** expressa for satisfeita, a execução deve seguir para as instruções do **bloco de comandos**
- ⌞ Após a execução da última instrução, o fluxo de execução segue para instrução após o delimitador de escopo fim-se
- ⌞ Caso a **condição** não seja satisfeita, o fluxo vai para a instrução após o delimitador

Condicional SE-ENTÃO em fluxograma

- Um desvio condicional SE-ENTÃO é um desvio condicional SE-ENTÃO-SENÃO no qual o bloco de comandos do senão não executa nada



Condicional ESCOLHA-CASO

```
escolha <variável>
  caso <valor 1> faça
    <bloco de comandos 1>
  caso <valor 2> faça
    <bloco de comandos 2>
  ...
  caso contrário faça
    <bloco alternativo>
fim-escolha
```

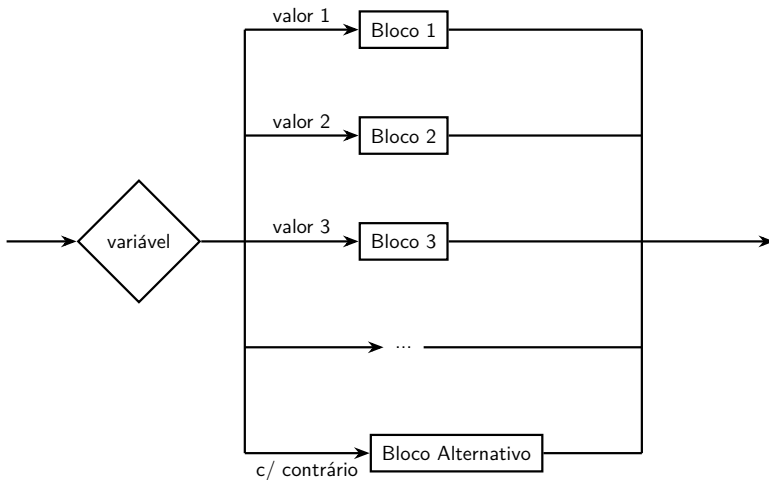
- ⌋ O comando ESCOLHA-CASO é uma variação em que o teste de igualdade é feito para uma única *variável*.

Condicional ESCOLHA-CASO

- ⌋ O comando ESCOLHA-CASO funciona como uma “instrução atalho” (açúcar sintático) para lidar com condicionais SE-ENTÃO-SENÃO aninhados ou SE-ENTÃO consecutivos em que a **condição** é de igualdade.

```
se <variável> = <valor 1> então
    <bloco de comandos 1>
fim-se
se <variável> = <valor 2> então
    <bloco de comandos 2>
fim-se
...
se <variável> != <valores anteriores> então
    <bloco alternativo>
fim-se
```

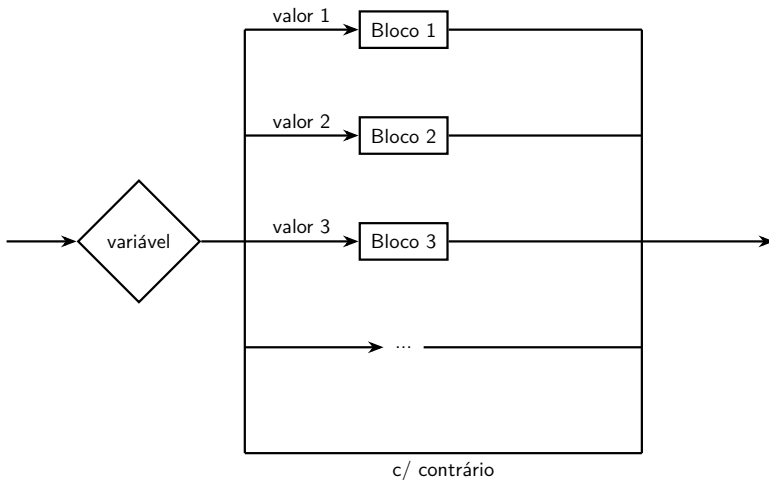
Condicional ESCOLHA-CASO em fluxograma



Condicional ESCOLHA-CASO

- ⌋ Se o trecho **caso contrário faça** for omitido na definição do desvio condicional, a linguagem de programação interpreta que, caso a **variável** não possua nenhum dos valores estabelecidos, nada deve ser executado
 - ⌋ Neste cenário o **bloco alternativo** é considerado vazio. O que, em termos de fluxograma, implica em omitir qualquer instrução no ramo de **caso contrário**
- ⌋○⌋ **IMPORTANTE:** o condicional ESCOLHA-CASO só lida com verificações de igualdade. Condições do tipo $var \geq n_0$ não podem ser utilizadas com essa estrutura condicional

Condicional ESCOLHA-CASO em fluxograma

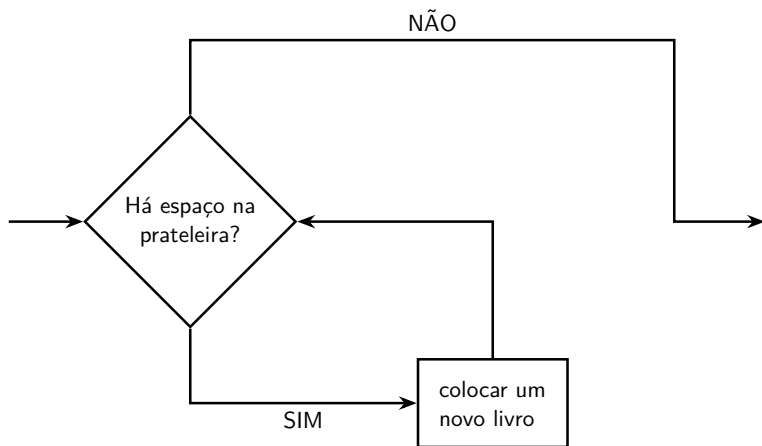


- ⌋ Neste cenário, você está arrumando livros em uma estante. Você deve encher uma prateleira de livros. Então, o que você faz?
- * Lembre-se que, em um algoritmo, as instruções precisam ser precisas e simples
 - * “Colocar os livros enquanto der” não é uma instrução precisa e simples
 - * Qual seria a instrução mais simples nesse cenário?

Estruturas de repetição

- ☾ Neste cenário, você está arrumando livros em uma estante. Você deve encher uma prateleira de livros. Então, o que você faz?
- * Lembre-se que, em um algoritmo, as instruções precisam ser precisas e simples
 - * “Colocar os livros enquanto der” não é uma instrução precisa e simples
 - * Qual seria a instrução mais simples nesse cenário?
- ☾ **Resposta:** Enquanto existir espaço na prateleira, coloque um livro nela. Perceba que, após colocar um livro, o ato de colocar um novo livro está **condicionado** pela existência de espaço suficiente para por o livro

Estrutura de repetição em fluxograma _____



Estrutura de repetição em pseudocódigo

- ☞ Tome **prateleira_cheia** como uma variável do tipo valor-verdade, de modo que **True** indica não haver espaço vago
- ☞ Em linguagens de programação, é muito comum usarmos ! para denotar uma operação de negação lógica
- ☞ Assim, a condição `!(prateleira_cheia)` é um equivalente semântico de “a prateleira não está cheia”

```
...  
enquanto !(prateleira_cheia) faça  
    colocar um livro na prateleira  
fim-enquanto  
...
```

Tipos de estrutura de repetição

- ⌋ Em termos de fluxograma, as estruturas de repetição sempre formarão um fluxo de execução circular que é controlado por um comando de decisão.
- ⌋ Os tipos mais comuns de estruturas de repetição nas linguagens de programação modernas são
 - * ENQUANTO-FAÇA
 - * FAÇA-ENQUANTO
 - * PARA
 - * PARA-CADA

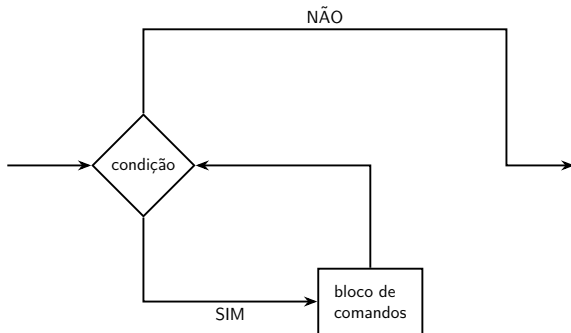
Repetição ENQUANTO-FAÇA

```
enquanto <condição> faça  
    <bloco de comandos>  
fim-enquanto
```

- ⌞ Se a **condição** expressa for satisfeita, a execução deve seguir para as instruções do **bloco de comandos**
- ⌞ Após a execução da última instrução, a condição é testada novamente
- ⌞ Caso a **condição** seja satisfeita, repete-se o processo
- ⌞ Caso contrário, o fluxo retorna para a instrução após o delimitador fim-enquanto

Repetição ENQUANTO-FAÇA em fluxograma

- Para garantir que essa repetição não seja infinita, é necessário que no bloco de comandos de repetição existam instruções que alterem os elementos envolvidos na condição



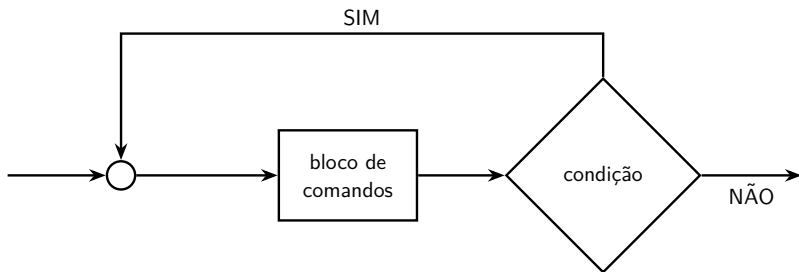
Repetição FAÇA-ENQUANTO

```
faça  
    <bloco de comandos>  
enquanto <condição>
```

- ⌞ Nesta estrutura de repetição, o **bloco de comandos** é executado obrigatoriamente ao menos uma vez
- ⌞ Após a execução do bloco, a **condição** é verificada
- ⌞ Se a **condição** for satisfeita, o fluxo é desviado para repetir o **bloco de comandos**
- ⌞ Caso contrário, a execução segue após testar a condição e verificar que não foi satisfeita

Repetição FAÇA-ENQUANTO em fluxograma —————

- ☾ O símbolo ○ no fluxograma é usado como um conector de fluxo
- ☾ Pode ser usado para demarcar pontos específicos da execução



Repetição PARA

```
para <var> = <início> até <fim> passo <inc> faça  
    <bloco de comandos>  
fim-para
```

- ⌋ Nesta estrutura de repetição, uma contagem é realizada enquanto a repetição é executada
- ⌋ No início, **var** recebe o valor **início**
- ⌋ Em seguida, verifica-se se o valor de **var** é menor ou igual ao valor **fim**
- ⌋ Em caso afirmativo, o **bloco de comandos** é executado

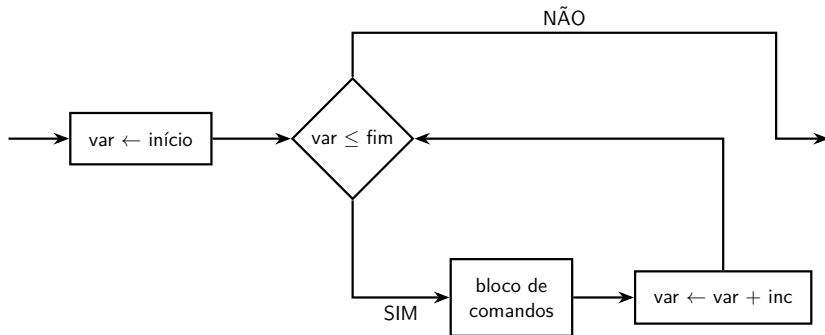
Repetição PARA

```
para <var> = <início> até <fim> passo <inc> faça  
    <bloco de comandos>  
fim-para
```

- ⌋ Ao final da execução do **bloco de comandos**, **var** é incrementada em **inc** unidades
- ⌋ Após o incremento, é verificado novamente se o valor de **var** é menor ou igual ao valor **fim**
- ⌋ E, com isso, mantém-se o fluxo de repetição através de uma contagem
- ⌋ Em pseudocódigo, quando o valor **inc** é $+1$ ou -1 , o trecho **passo <inc>** é omitido no comando

Repetição PARA em fluxograma

- ☾ A atualização do valor de **var** sinalizado no fluxograma é feito de maneira automática pelo comando



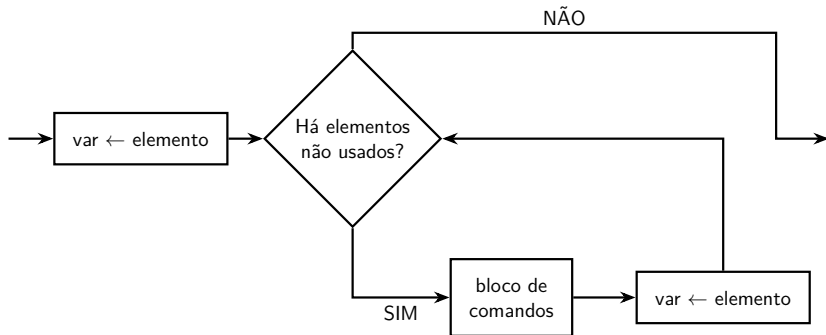
Repetição PARA-CADA

```
para cada <var> em <coleção> faça  
    <bloco de comandos>  
fim-para
```

- ⌈ **var** assume, a cada nova passagem da repetição, um dos valores da **coleção**
- ⌈ A condição de parada é que todos os elementos da coleção sejam usados
- ⌈ As **coleções** em questão são sempre finitas, logo esta é uma estrutura de repetição definida

Repetição PARA-CADA em fluxograma

- ☾ A atualização do valor de **var** sinalizado no fluxograma é feito de maneira automática pelo comando



Exercício

1. Para cada um dos cenários a seguir, apresente um algoritmo em pseudocódigo e em fluxograma
 - (a) Leia 3 números e diga qual a média, a soma e o produto dos 3 números
 - (b) Leia 20 números e diga qual foi o maior número lido
 - (c) Leia um número inteiro positivo n e, em seguida, leia n números e diga qual é o maior deles
 - (d) Leia um número inteiro positivo n e imprima na tela todos os números naturais pares menores ou iguais a n

